

UNIVERSITÀ TELEMATICA PEGASO

Corso di laurea triennale in
INFORMATICA PER LE AZIENDE DIGITALI L-31

Tesi di laurea in:

MACHINE LEARNING APPLICATO ALLO SMART TICKETING:

Dalla generazione dati tramite LLM multi-agent allo smistamento automatico dei ticket

Gabriele Stagnitta
Matricola: 0312401645

Anno Accademico
2025/2026

"What's your ETA?
Chill out, man, we're right on time."
– Twenty One Pilots, *Midwest Indigo*

Indice

0 Parte Zero - Introduzione **1**
0.1 Descrizione situazione - Triage automatico dei ticket con Machine Learning . . . 1

1 Parte Prima - Descrizione del processo **3**
1.1 Utilizzo delle conoscenze e abilità derivate dal percorso di studio . 3
1.2 Fasi di lavoro e relativi tempi di implementazione 5

2 Parte Seconda - Predisposizione dell'elaborato **17**
2.1 Obiettivi del progetto 17
2.2 Contestualizzazione 20
2.3 Descrizione dei principali aspetti progettuali 21
2.3.1 Generazione Dataset elementare 21
2.3.2 Generazione Dataset con Gemma4:e2b 25
2.3.3 Generazione Dataset con architettura multi-agent 28
2.3.4 Pipeline Machine Learning 28
2.3.5 Dashboard 36
2.4 Campi di applicazione 38
2.5 Valutazione dei risultati 40

Elenco delle figure

1.1 Pipeline dell'architettura multi-agent 8
1.2 Caratteristiche Hardware del computer responsabile della generazione 8
1.3 Matrice di confusione - classificazione categoria dataset pseudorandom 10
1.4 Matrice di confusione - valutazione priorità dataset gemma4_unbalanced 11
1.5 Matrice di confusione - valutazione priorità dataset gemma4_balanced 12
1.6 Matrice di confusione - valutazione priorità dataset gemma4e2b . 13
1.7 Matrice di confusione - valutazione priorità dataset multi-agent . 14
1.8 Interfaccia grafica della web app 15

2.1 Flusso di lavoro del Project Work 18
2.2 Estratto del contenuto del file lessico.json 22
2.3 Implementazione della generazione procedurale con pesi distribuiti e iniezione di rumore semantico 23
2.4 Esempio di struttura base dei parametri di generazione realizzata ereditando la classe BaseModel di Pydantic 26
2.5 Funzione della generazione di un ticket. Qui viene formattato il prompt per la creazione di un ticket e sottoposto all'LLM. 26
2.6 Ciclo responsabile della generazione di n ticket da generare 27
2.7 Esempio di struttura base dei parametri di revisione realizzata ereditando la classe BaseModel di Pydantic 28
2.8 Grafico a torta della distribuzione dei ticket per categoria 29
2.9 Tabella incrociata della composizione priorità nelle varie categorie 29
2.10 Violin plot + Swarm Plot della distribuzione del numero di parole per Priorità e Categoria 30
2.11 Grafico a barre delle 15 parole più frequenti nel dataset 30
2.12 Lista di supporto per le stopword 31
2.13 Funzione remove_key_word 31
2.14 Formule matematiche per il calcolo del peso TF-IDF 32
2.15 Approccio gerarchico alla previsione della categoria e priorità . . . 33
2.16 Script responsabile al salvataggio di modelli e vettorizzatori 34
2.17 Top 15 Parole più influenti per la categoria 35
2.18 Top 15 Parole più influenti per la priorità 35
2.19 Caricamento modelli 36
2.20 Logica dell'analisi del singolo ticket. Le funzioni clean_text e get_top_words sono le stesse analizzate anche nel notebook 03. . 36

2.21 Interfaccia della dashboard - sezione “elaborazione batch” 37
2.22 Mockup di un modello di Customer Care, dove sono presenti delle richieste verosimili da parte di genitori immaginari 38
2.23 Mockup di un modello di Call Center per un’azienda del settore primario. Le chiamate vengono smistate tra gli operatori in base al reparto di competenza. 39
2.24 Esempio di flusso di gestione ticket con Machine Learning. In verde i processi umani/preesistenti, in arancione innovazione Machine Learning e in azzurrino output aziendale. 40

Elenco delle tabelle

1.1 Esempio di ticket generati tramite algoritmo di generazione pseudorandom 7
1.2 Esempio di ticket generati tramite algoritmo avanzato utilizzando Gemma4 7
1.3 Esempio di ticket generati tramite algoritmo di generazione avanzato utilizzando Gemma4:e2b 7
1.4 Esempio di ticket generati tramite architettura multi-agent 8
1.5 Analisi dei migliori risultati degli algoritmi in base al dataset . . . 9
1.6 Classification report - valutazione categoria dataset pseudorandom (Linear SVC) 10
1.7 Classification report - valutazione priorità dataset gemma4_unbalanced (Linear SVC) 11
1.8 Classification report - valutazione priorità dataset gemma4_balanced (Random Forest Classifier) 12
1.9 Classification report - valutazione priorità dataset gemma4e2b (Approccio gerarchico Random Forest) 13
1.10 Classification report - valutazione priorità dataset multi-agent (Linear SVC) 14

2.1 Esempio di classification report 18

Capitolo 0

Introduzione

0.1 Descrizione situazione - Problema del Triage automatico dei ticket con Machine Learning

Un'azienda riceve ogni giorno decine di **ticket** via email o modulo web: richieste su fatture e pagamenti **(Amministrazione)**, segnalazioni di malfunzionamenti **(Tecnico)**, domande su offerte e ordini **(Commerciale)**. Lo smistamento manuale rallenta i tempi di risposta e crea frustrazione.

L'obiettivo del project work è realizzare un **prototipo essenziale e riproducibile** che, a partire da un piccolo dataset sintetico di **ticket brevi (oggetto + descrizione)**, classifichi automaticamente ogni ticket nella categoria più pertinente e suggerisca una **priorità operativa** (*bassa*/*media*/*alta*).

Si richiede l'uso di tecniche di ML di base mantenendo il focus su semplicità del codice e chiarezza del flusso: **input testuale, output con categoria e priorità**, e una misura elementare di qualità (ad esempio accuracy o F1) per verificare che il modello sia **ragionevolmente affidabile**. Tutti i dati devono essere generati ad hoc, senza informazioni personali, privilegiando un design minimale che permetta di eseguire e comprendere il prototipo in pochi minuti.

Capitolo 1

Descrizione del processo

1.1 Utilizzo delle conoscenze e abilità derivate dal percorso di studio

La realizzazione di questo *Project Work* ha permesso l'integrazione tra le conoscenze multidisciplinari maturate nel percorso di studi e quelle acquisite in ambito professionale. Lavorando come insegnante per corsi online di *coding* per studenti tra i 7 e i 18 anni e come responsabile dei corsi di `Python` e di Intelligenza Artificiale con `Python`, ho potuto applicare sia in fase di stesura dei vari *Jupyter Notebook* che nella scrittura di questo documento diverse metodologie pratiche che utilizzo giornalmente per la composizione dei materiali didattici. Inoltre ho potuto utilizzare alcune competenze trasversali per ottimizzare i processi di sviluppo, implementazione e analisi delle fasi.
Dal punto di vista accademico, ho avuto modo di lavorare al progetto in contemporanea alla frequenza del corso di **Programmazione 2** del **Prof. Claudio Tommazoli**. Questo mi ha permesso di avvalermi delle competenze apprese sull'utilizzo di librerie come `pandas`, `seaborn`, ecc... Difatti, come verrà meglio analizzato in questo documento, ho più volte avuto modo di utilizzare questi strumenti per l'*EDA* (*Exploratory Data Analysis*). Altra cosa utile appresa nel corso è sicuramente la libreria `Tkinter`, che non conoscevo prima di seguire l'insegnamento, utilizzata per un approccio iniziale per la creazione della *Dashboard* finale, nonostante alla fine abbia deciso di utilizzare un *framework* più moderno, ovvero `niceGUI`.
Proprio in merito all'*EDA* e analisi dei dati visualizzati, ho trovato fondamentali i concetti appresi studiando il corso di **Calcolo delle Probabilità e Statistica** tenuto dal **Prof. Sergio Frigeri**. Difatti le capacità analitiche apprese nel corso mi hanno permesso di interpretare con maggiore consapevolezza i dati generati e gli *output* della *pipeline* di ML. Queste conoscenze hanno inoltre permesso di soddisfare la mia curiosità sul funzionamento di alcuni algoritmi matematici essenziali di `scikit-learn`, come ad esempio il `TfidfVectorizer` (ragionamento matematico che citerò nell'elaborato).
Una menzione d'onore va infine al corso di **Algoritmi e Strutture Dati** del **Prof. Stefano D'Urso**, al corso di **Ingegneria del Software** tenuto dai **Docenti Fabiano Pecorelli, Massimiliano Pirani, Roberto Vergallo e**

Martina Iammarino, e al corso di **Tecnologie Web**, che ho seguito nella mia precedente Università. I primi due insegnamenti mi hanno aiutato a immaginare la struttura del codice non solo come una sequenza di istruzioni, ma come un'architettura software strutturata e stratificata, dove ogni istruzione va ponderata per garantire la massima efficienza e manutenibilità in caso di modifiche durante il ciclo di vita del progetto (e se ne sono presentate non poche). Le competenze acquisite in **Tecnologie Web** invece si sono rivelate determinanti nello sviluppo di un'interfaccia utente (UI) moderna e funzionale al progetto.

1.2 Fasi di lavoro e relativi tempi di implementazione

La realizzazione dell'elaborato è stata organizzata in fasi strutturate per ottimizzare il *workflow*: dall'analisi delle procedure per la creazione di un dataset sintetico fino al *deploy* di una *web app* come risultato finale. I vari *Notebook* consultabili nel *github* sono stati creati e compilati sequenzialmente, in modo da mantenere l'ordine delle fasi di lavoro.

1. **Fase di analisi e ricerca:** La prima fase è stata quella di analisi del *project work*. Non era la prima volta che mi cimentavo nello sviluppo di algoritmi di *machine learning*, però non avevo mai affrontato il problema in ambito accademico. Ho dunque ricercato sul web (in maniera particolare nella famosa piattaforma `Kaggle.com`) lo sviluppo di problemi simili, come ad esempio problemi di *Natural Language Processing* applicati a vari contesti, con lo scopo di apprendere nuove tecniche da poter considerare per il *Project Work*.

2. **Sviluppo del progetto:** Una volta finito il penultimo esame, ovvero nel mese di Giugno 2026, ho iniziato lo sviluppo del progetto, suddiviso in tre grandi macro-sezioni.

Generazione Dataset *(Tempo di realizzazione: circa 2 settimane)* – Relativo ai *Jupyter Notebook* `01_Generazione_Dataset_Elementare.ipynb`, `02.1_Generazione_Dataset_Avanzato_Con_Gemma.ipynb` e `02.2_Generazione_Dataset_Architettura_Actor_Critic.ipynb`.

Questa fase dava l'illusione di essere stata la più rapida da portare a termine, ma ho dovuto riprenderla a seguito di un primo sviluppo della *pipeline* di *Machine Learning* in quanto i risultati non erano soddisfacenti. I ticket, inizialmente generati pseudo-casualmente partendo da un file `.json` con al suo interno diverse forme lessicali base, contenevano delle frasi spesso ripetitive, sconnesse e/o senza senso. Riflettevo già in questa fase dei possibili problemi causati dalla natura poco dinamica dell'algoritmo. Il file generato è identificato come `ticket_pseudorandom.csv` (vedi Tabella 1.1).

A questo punto, ho effettivamente iniziato a lavorare alla *pipeline* ML, e le criticità ipotizzate si sono confermate: il dataset portava gli algoritmi in *overfitting*: sebbene il *classification report* riportasse dei risultati ottimi (come verrà analizzato a breve da Figura 1.3), il modello si è rivelato fragile nella generalizzazione. Ho dunque provato la generazione del dataset con LLM commerciali come Gemini e ChatGPT, ma il risultato era troppo semplicistico e soprattutto poco accademico.

Ho optato per qualcosa di più professionale, ovvero un'architettura che generasse dataset artificiali utilizzando un LLM in locale: `Gemma4`, nello specifico. Questo processo ha richiesto un paio di giorni per essere perfezionato a causa della sua natura più complessa (anche dal punto di vista del livello hardware) e dei *trial and error* necessari fino al raggiungimento di un perfetto equilibrio per la generazione dei dati: è sì stato utilizzato un LLM, ma che ha lavorato in locale e

alle mie condizioni. Non volevo che venisse generato un dataset per intero, bensì un ticket per volta.

La prima sfida (i primi 2-3 giorni) è stata il fatto che il modello base di `Gemma4` creasse dei ticket veramente realistici, ma al tempo stesso troppo complessi per gli algoritmi di ML, senza considerare i tempi di elaborazione troppo lunghi; il tempo di generazione medio era di circa 2 ticket al minuto, il che significava circa 120 ticket l'ora e di conseguenza circa 4 ore per la generazione di un dataset con 500 ticket (`ticket_gemma4_balanced.csv`, vedi Tabella 1.2). Un risultato più bilanciato e sostenibile (*output* di qualità leggermente inferiore, ma con tempi di elaborazione più che dimezzati) lo devo ad un modello di Gemma più leggero, ovvero `Gemma4:e2b`. Il tempo di elaborazione è sceso a circa 5 ticket ogni minuto, dunque meno di 2 ore per la creazione di un dataset di 500 ticket. Questo ha permesso di effettuare molte più prove e di modificare più volte la struttura del *prompt* base, rendendolo in grado di reagire sempre in modi diversi ai parametri che lo generano (`ticket_gemma4e2b.csv`, vedi Tabella 1.3).

L'ultima evoluzione di questa fase ha previsto l'implementazione di un'architettura Multi-Agente chiamata in letteratura *Actor-Critic*. Invece di impostare l'intero carico di lavoro su un'unica LLM, si crea un sistema in cui i modelli "ragionano" e si correggono a vicenda (vedi Figura 1.1). Nello specifico, l'architettura è stata così divisa:

- **Agente 1 (Il Generatore):** Riceve i parametri casuali (categoria, priorità, ecc.) e ha il compito di generare la prima bozza del ticket. A questa fase è stata inoltre aggiunta una variabile per la generazione casuale del genere del mittente (per variare rispetto allo standard maschile del modello).

- **Agente 2 (Il Revisore):** Riceve in input il testo generato dall'Agente 1 e un *prompt* di validazione molto rigido (es. *"Agisci come un revisore qualità..."*). Restituisce un nuovo ticket dove corregge eventuali artefatti senza snaturare il tono dell'originale.

I ticket generati sono anche loro consultabili all'interno del GitHub, oppure vedi Tabella 1.4.

NOTA: il tempo di elaborazione può variare da dispositivo a dispositivo. In Figura 1.2 è possibile visionare le caratteristiche hardware della macchina in cui è stata effettuata questa parte di progetto.

Ticket	Categoria	Priorità
Ciao, per favore dovremmo proporre i dati del nuovo lead per un potenziale cliente il prima possibile. Fammi sapere se ci sono problemi. Grazie!	Commerciale	Alta
Ciao, per favore dovremmo ripristinare un bug bloccante durante il login il prima possibile.Fammi sapere si ci sono problemi. Grazie!	Tecnico	Alta

Tabella 1.1: Esempio di ticket generati tramite algoritmo di generazione pseudorandom

Ticket	Categoria	Priorità
Così non si può lavorare! Guardate che questi costi sono assurdi, mi state prendendo in giro? Ho provato a fare un preventivo ieri ed è diverso da quello di oggi. Ditemi voi cosa sta succedendo	Commerciale	Alta
Non so più che fare... Dunque, guardate qui: mi sembra di aver cliccato su qualcosa e ora questa schermata è bloccata da un sacco di tempo. Non ho idea se sia un mio errore o se il sistema stia semplicemente morendo davanti ai miei occhi. Aiuto, per favore.	Tecnico	Basso

Tabella 1.2: Esempio di ticket generati tramite algoritmo avanzato utilizzando Gemma4

Ticket	Categoria	Priorità
Ho un piccolo dubbio sulla configurazione della mia rete Wi-Fi. Vorrei capire come ottimizzare il routing senza causare problemi. Potete darmi qualche dritta su come fare questo setup in modo più efficiente?	Tecnico	Bassa
Ho notato un errore nella fattura di questo mese. Vorrei sapere come procedere per la correzione.	Commerciale	Bassa

Tabella 1.3: Esempio di ticket generati tramite algoritmo di generazione avanzato utilizzando Gemma4:e2b

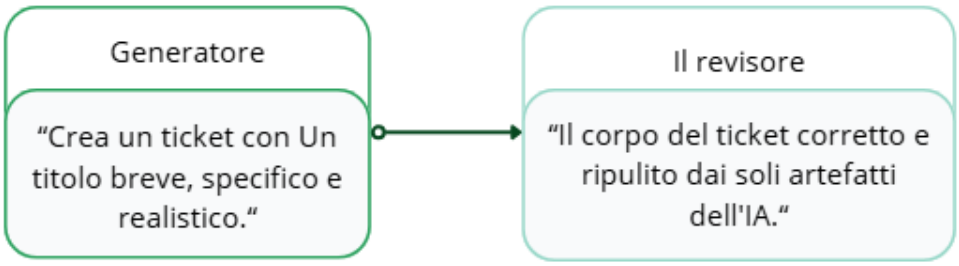


Figura 1.1: Pipeline dell'architettura multi-agent

Ticket	**Categoria**	**Priorità**
Ciao, ho notato un'anomalia nel setup del report mensile. Per favore, risolvete questo problema senza che io debba fare troppi passaggi inutili. Spero sia una cosa semplice da gestire.	Amministrazione	Media
Ciao, vi scrivo perché ho notato un problema con un addebito commerciale. Sono sospettoso sul saldo attuale e avrei bisogno che controllaste subito il nostro account. Vorrei capire cosa è successo velocemente.	Commerciale	Medio

Tabella 1.4: Esempio di ticket generati tramite architettura multi-agent

Processore	12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)
RAM Installata	16.0 GB
Scheda grafica	NVIDIA GeForce RTX 4050 Laptop GPU (6 GB)

Figura 1.2: Caratteristiche Hardware del computer responsabile della generazione

Pipeline Machine Learning *(Tempo di realizzazione: circa 3 settimane)* – Relativo al *Notebook* `03_Pipeline_Machine_Learning.ipynb`.

Questa è stata la fase dove mi sono soffermato maggiormente e anche la più sfidante. Nonostante avessi già delle idee sul tipo di algoritmi da utilizzare, la sfida principale è stata quella di riuscire ad avere un risultato ben bilanciato con i miei dati. Questo perché non tutti i dataset generati riuscivano a dare dei risultati soddisfacenti. È possibile consultare una migliore analisi dei risultati all'interno della fase 6 del *notebook*. A seguire vengono presentati i *classification report* e l'analisi dei migliori risultati dei vari dataset sintetizzati (Tabella 1.5).

Dataset	**Accuracy Categoria**	**Accuracy Priorità**
pseudorandom	0.89 con LinearSVC	0.86 RandomForest
gemma4_unbalanced	0.73 con LinearSVC	0.54 con Approccio Gerarchico
gemma4_balanced	0.82 con LinearSVC	0.53 con Approccio Gerarchico
gemma4e2b	0.84 con Random Forest	0.73 con Approccio Gerarchico
actor_critic	0.79 con LinearSVC	0.77 con LinearSVC
unione_di_tutti	0.93 con LinearSVC	0.80 con LinearSVC

Tabella 1.5: Analisi dei migliori risultati degli algoritmi in base al dataset

Iniziamo con i risultati del dataset realizzato pseudo-randomicamente. Come accennato prima, a fronte di risultati eccellenti, ho sviluppato uno script che mi permettesse di analizzare un ticket in input (prototipo della futura *dashboard* finale), ma reagiva piuttosto male. Come previsto, utilizzando le parole nella stessa forma utilizzata durante la creazione, l'algoritmo classificava quasi sempre correttamente i ticket. Cambiando però il contesto, perdeva le sue capacità di generalizzare (Figura 1.3 e Tabella 1.6).

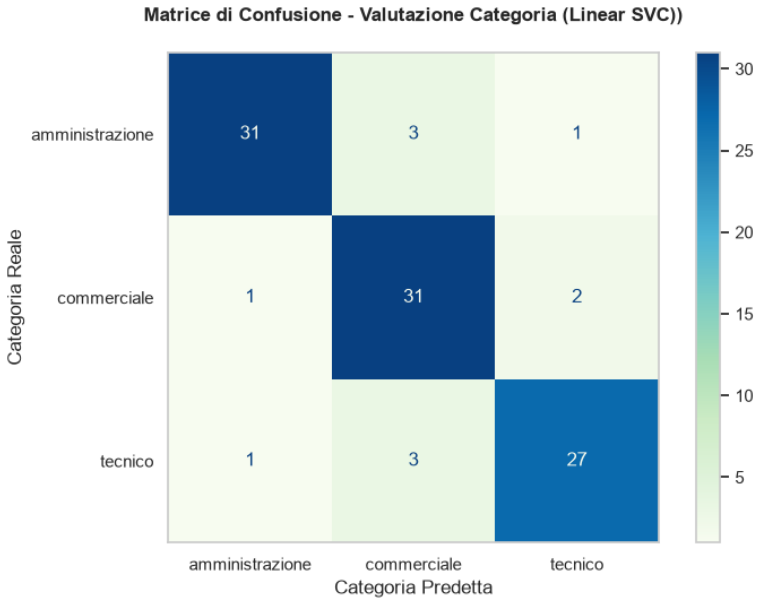


Figura 1.3: Matrice di confusione - classificazione categoria dataset pseudorandom

	precision	**recall**	**f1-score**	**support**
amministrazione	0.94	0.89	0.91	35
commerciale	0.84	0.91	0.87	34
tecnico	0.90	0.87	0.89	31
accuracy			0.89	100
macro avg	0.89	0.89	0.89	100
weighted avg	0.89	0.89	0.89	100

Tabella 1.6: Classification report - valutazione categoria dataset pseudorandom (Linear SVC)

Nutrivo grandi speranze per i dataset generati dalla versione standard di `Gemma4`, ma invece la matrice di confusione evidenzia uno schema problematico. Con un'*accuracy* di 0.51, possiamo considerare questi risultati niente di più rispetto al lancio di una moneta. I maggiori imputati a questi problemi sono: frasi troppo complesse e ricche di vocaboli per algoritmi di *machine learning* semplici, pochi dati in input (500 ticket totali) e un dataset creato volontariamente sbilanciato sul numero di ticket per priorità allo scopo di simulare la vita reale (Figura 1.4 e Tabella1.7).

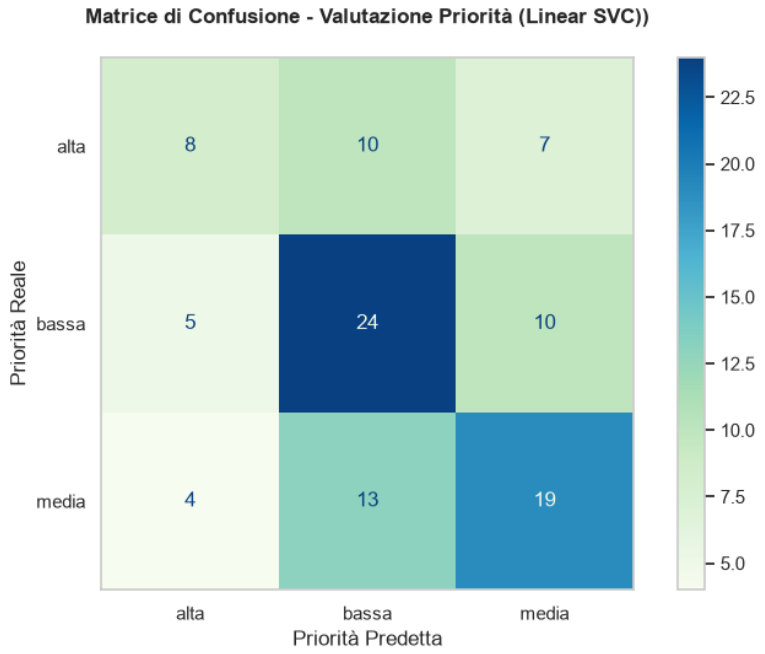


Figura 1.4: Matrice di confusione - valutazione priorità dataset gemma4_unbalanced

	precision	**recall**	**f1-score**	**support**
alta	0.47	0.32	0.38	25
bassa	0.51	0.62	0.56	39
media	0.53	0.53	0.53	36
accuracy			0.51	100
macro avg	0.50	0.49	0.49	100
weighted avg	0.51	0.51	0.50	100

Tabella 1.7: Classification report - valutazione priorità dataset gemma4_unbalanced (Linear SVC)

Ho provato a risolvere il terzo problema generando altri ticket bilanciando la priorità, ma i risultati con l'algoritmo *Random Forest Classifier* sono addirittura peggiorati, con un'*accuracy* scesa a 0.39 (Figura 1.5 e 1.8).

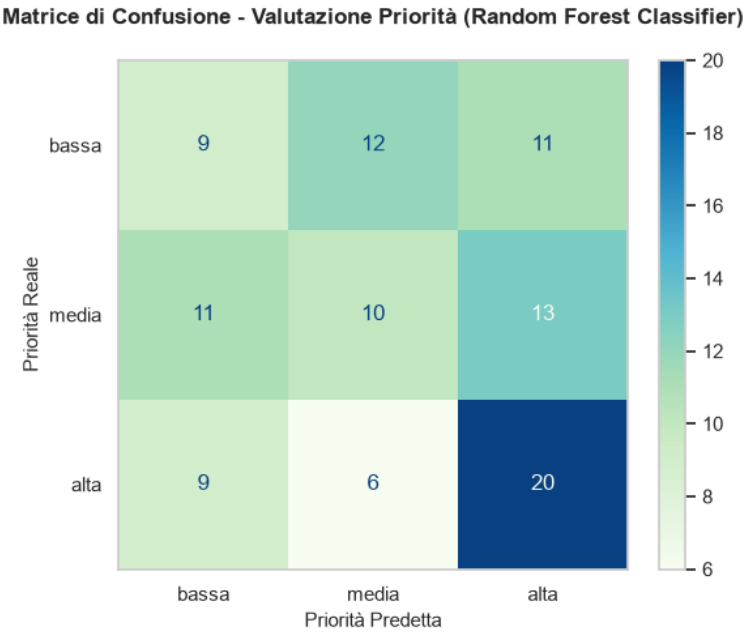


Figura 1.5: Matrice di confusione - valutazione priorità dataset gemma4_balanced

	precision	**recall**	**f1-score**	**support**
alta	0.45	0.57	0.51	35
bassa	0.31	0.28	0.30	32
media	0.36	0.29	0.32	34
accuracy			0.39	101
macro avg	0.37	0.38	0.37	101
weighted avg	0.38	0.39	0.38	101

Tabella 1.8: Classification report - valutazione priorità dataset gemma4_balanced (Random Forest Classifier)

Ho svolto un ulteriore esperimento sfruttando il modello `Gemma4:e2b`. La sua leggerezza, dovuta al minor numero di parametri e al vocabolario meno complesso, si è rivelata una soluzione eccellente. In combinazione ad un'operazione di *stemming* durante il *preprocessing* e ad un approccio gerarchico dell'analisi, i risultati sono nettamente migliorati (Figura 1.6 e 1.9).

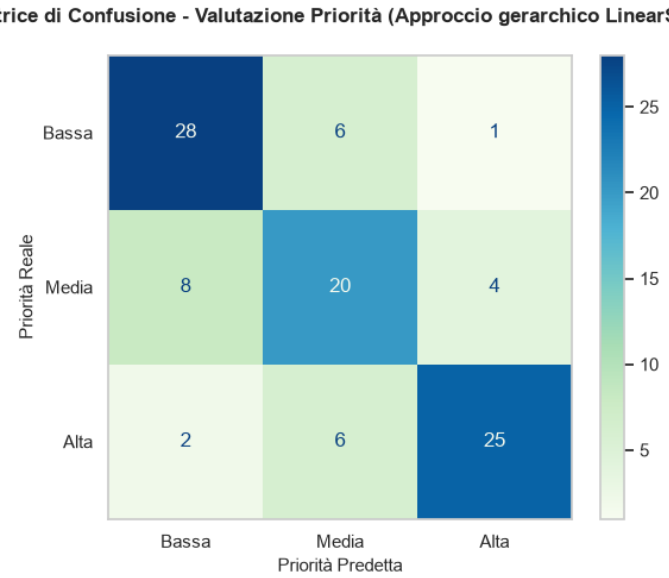


Figura 1.6: Matrice di confusione - valutazione priorità dataset gemma4e2b

	precision	**recall**	**f1-score**	**support**
Alta	0.76	0.85	0.80	33
Bassa	0.68	0.54	0.60	35
Media	0.51	0.56	0.54	32
accuracy			0.65	100
macro avg	0.65	0.65	0.65	100
weighted avg	0.65	0.65	0.65	100

Tabella 1.9: Classification report - valutazione priorità dataset gemma4e2b (Approccio gerarchico Random Forest)

Il bilanciamento definitivo lo ha dato infine il dataset creato con **l'architettura Multi-Agente**, che ha dimostrato una maggiore stabilità nella generalizzazione dei ticket, funzionando molto bene anche su validazioni con testi "umani" scritti a mano (Figura 1.7 e 1.10).

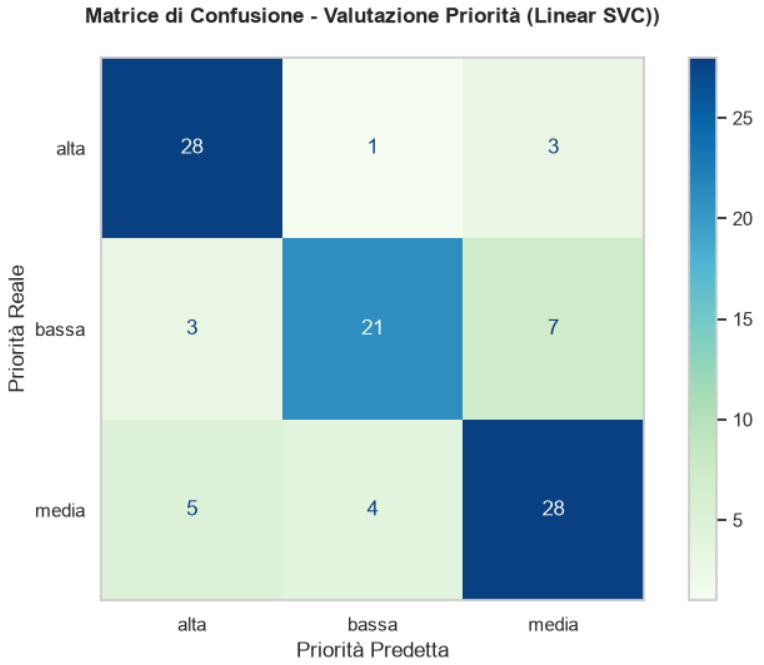


Figura 1.7: Matrice di confusione - valutazione priorità dataset multi-agent

	precision	**recall**	**f1-score**	**support**
alta	0.78	0.88	0.82	32
bassa	0.81	0.68	0.74	31
media	0.74	0.76	0.75	37
accuracy			0.77	100
macro avg	0.77	0.77	0.77	100
weighted avg	0.77	0.77	0.77	100

Tabella 1.10: Classification report - valutazione priorità dataset multi-agent (Linear SVC)

In conclusione, il problema più grande affrontato in questa fase è stato trovare risultati soddisfacenti per la valutazione delle priorità, unico punto in cui gli algoritmi di *machine learning* utilizzati hanno mostrato maggiori debolezze.

Creazione di una web app per la dashboard *(Tempo di realizzazione: circa 2 giorni)* – Relativo al file `Dashboard.py`.

Questa è stata la sezione più leggera da elaborare dell'intero progetto. Inizialmente avevo optato per una *UI* sviluppata tramite `Tkinter` ma la resa grafica non era soddisfacente. La scelta è ricaduta su `NiceGUI`, un *framework* per l'*UI* visualizzato su interfaccia web (Figura 1.8). Una volta testato in locale, ho impiegato circa un'ora per effettuare un *deploy* su Render per rendere la *web app* di facile accesso (`https://project-work-vdxq.onrender.com/`).

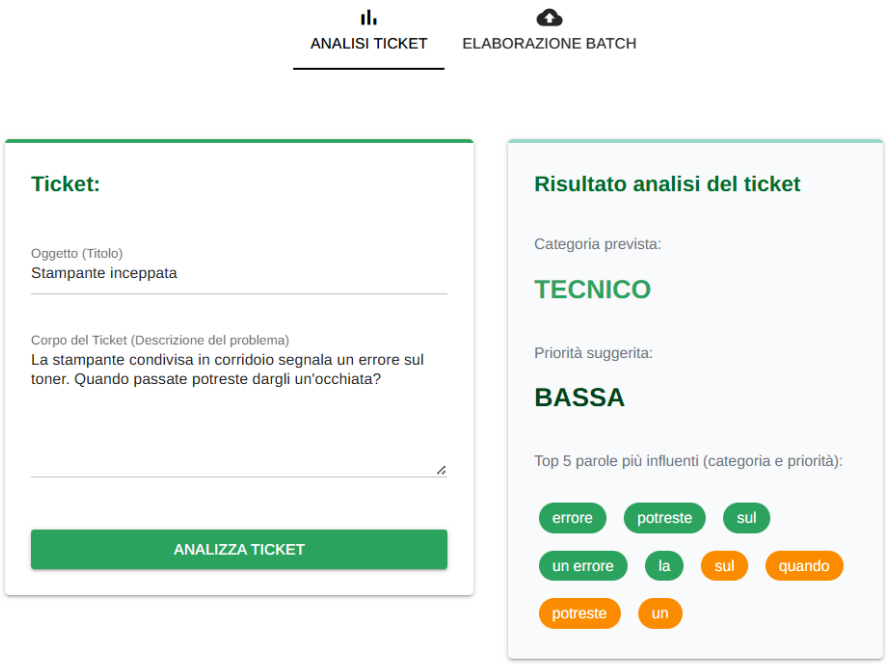


Figura 1.8: Interfaccia grafica della web app

Scansiona il codice QR per accedere direttamente alla web app.

Capitolo 2

Predisposizione dell'elaborato

2.1 Obiettivi del progetto

Il *Project Work* pone l'obiettivo primario di progettare, sviluppare e validare un prototipo software essenziale, modulare e completamente riproducibile per l'automatizzazione del *triage* di ticket, seguendo le specifiche richieste dalla traccia PW18.

Descrizione della traccia:

> "Un'azienda riceve ogni giorno decine di ticket via email o modulo web [...] Lo smistamento manuale rallenta i tempi di risposta e crea frustrazione. L'obiettivo del project work è realizzare un prototipo essenziale e riproducibile che [...] classifichi automaticamente ogni ticket nella categoria più pertinente e suggerisca una priorità operativa [...]."

L'elaborato riesce a trasformare un problema operativo tipico degli *help desk* (il collo di bottiglia generato dalla lettura e dallo smistamento manuale di comunicazioni non strutturate) in una *pipeline* di *Natural Language Processing* (NLP) efficiente che automatizza le procedure.

Nel dettaglio, il lavoro ha permesso di raggiungere i seguenti macro-obiettivi:

1. **Smistamento mirato sulle aree di business aziendali:** Come richiesto dalle specifiche, il sistema prende in *input* un testo grezzo e restituisce in *output* una classificazione automatica del ticket:
   - **Assegnazione della Categoria** → smistamento verso il reparto competente (Amministrazione, Tecnico o Commerciale)
   - **Suggerimento della Priorità** → valutazione dell'urgenza per ottimizzare i tempi di gestione su ticket critici.
2. **Generazione dei dati "ad hoc":** Seguendo i vincoli di generazione "senza informazioni personali" e partendo da un "piccolo dataset sintetico di ticket brevi (oggetto + descrizione)", l'elaborato ha raggiunto l'obiettivo di creare dataset completamente artificiali ma semanticamente verosimili.

Sfruttando la generazione avanzata tramite modelli LLM installati in locale (`Gemma 4:e2b`) e controllati tramite schemi *Pydantic*, sono stati generati ticket strutturati esattamente in Titolo e Corpo del messaggio, privi di qualsiasi dato sensibile ma ricchi del lessico e delle sfumature necessarie all'addestramento.

3. **Semplicità del codice e chiarezza di flusso:** Rispondendo al requisito "tecniche di ML di base mantenendo il focus su semplicità e chiarezza", l'architettura è stata sviluppata utilizzando semplici algoritmi di classificazione supervisionati, ma comunque altamente performanti per il NLP: `LinearSVC` e `RandomForestClassifier`, utilizzando `TfidfVectorizer` per la vettorizzazione.

   Il flusso di lavoro risulta dunque veramente semplice ed intuitivo.

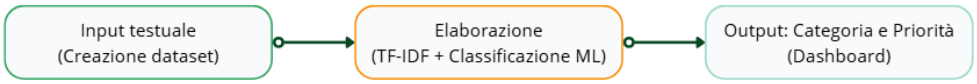


Figura 2.1: Flusso di lavoro del Project Work

4. **Verifica dell'affidabilità:** Per verificare che il modello fosse "ragionevolmente affidabile", ogni esperimento condotto nei *Jupyter Notebook* è stato accompagnato da *Classification Report* (utilizzo di *Accuracy*, *F1-score* e Matrici di Confusione) che permettono di validare scientificamente il prototipo.

	precision	**recall**	**f1-score**	**support**
amministrazione	0.72	0.81	0.76	32
commerciale	0.73	0.69	0.71	35
tecnico	0.94	0.88	0.91	33
accuracy			0.79	100
macro avg	0.79	0.79	0.79	100
weighted avg	0.79	0.79	0.79	100

Tabella 2.1: Esempio di classification report

5. **Riproducibilità e design minimale per un'esecuzione rapida:** La caratteristica "un prototipo essenziale e riproducibile" e che "permetta di eseguire e comprendere il prototipo in pochi minuti" viene soddisfatta dalla stesura dei *Jupyter Notebook* ampiamente commentati, eseguibili in blocco su qualsiasi computer in locale o in cloud (ad esempio Google Colab), e dal *deploy* di una *web app* per la *dashboard*, facilmente fruibile tramite qualsiasi browser web senza necessità di alcuna installazione o configurazione tecnica.

NOTA: l'unico elemento che potrebbe creare rallentamenti riguardo la riproducibilità è il notebook per la creazione del dataset usando Gemma, a causa della sua maggiore complessità sia progettuale che di elaborazione. Ho deciso di non rimuovere la prima versione del codice anche per questo motivo. All'interno della cartella `dataset` *sono disponibili diversi* `.csv` *generati pronti all'uso. Link alla cartella:* `https://github.com/gabrieleStagnitta/Project-Work/tree/main/dataset`

2.2 Contestualizzazione

Contesto teorico:

Dal punto di vista scientifico, l'elaborato si inserisce nel dominio del *Natural Language Processing* (NLP) e della Classificazione Testuale Supervisionata.

Storicamente parlando, i primi sistemi di smistamento automatico dei testi si basavano su approcci statici con regole basate su parole chiave. Sistemi molto semplici da implementare, però al tempo stesso molto fragili alla complessità del linguaggio naturale. Il contesto teorico di questo elaborato si fonda proprio sul superamento di tali limiti tramite il *Machine Learning* statistico, in cui gli algoritmi comprendono in maniera supervisionata i *pattern* semantici e le relazioni con il gruppo di appartenenza.

Nonostante la letteratura accademica moderna sia dominata da architetture *Deep Learning*, *Transformer*, ecc... come ad esempio BERT (*Bidirectional Encoder Representations from Transformers*), oppure `fastText`, per *task* di classificazione di testi brevi, algoritmi di *Machine Learning* tradizionali rappresentano comunque un'ottima soluzione. In maniera particolare, gli algoritmi utilizzati sono il `LinearSVC` e `RandomForestClassifier`, con il sostegno di `TfidfVectorizer` per l'estrazione delle *feature*.

Un ulteriore pilastro teorico del progetto è l'utilizzo di LLM (nel caso specifico `Gemma4:e2b`) non come strumento di generazione del dataset per intero, bensì come generazione atomica dei ticket seguendo *prompt* generati casualmente in *runtime*. Ritengo questo elemento un grosso punto di forza dell'intero progetto in quanto è stato essenziale per rispondere ad un problema classico del *Machine Learning*, ovvero l'assenza di dati etichettati di alta qualità in contesti creati da zero, cosa che in fase di progettazione stava inizialmente creando non pochi problemi.

Contesto applicativo:

Dal punto di vista applicativo, il progetto si inserisce nel settore del CRM e dei sistemi di *Smart Ticketing*.

Nelle aziende di media-larga dimensione, la ricezione quotidiana di comunicazioni non strutturate tramite diversi canali può essere un problema. Lo smistamento manuale genera colli di bottiglia che rallentano il lavoro degli operatori e lo espongono a potenziali errori di assegnazione. Inoltre implica del lavoro umano ripetitivo e altamente alienante.

In uno scenario reale, l'elaborato avrebbe il compito di agire come da "primo filtro", che si fa carico proprio del processo ripetitivo descritto nel punto precedente e lo trasforma in un flusso di lavoro misurabile e tracciabile, lasciando all'operatore il solo compito di supervisione e risoluzione effettiva del problema.

2.3 Descrizione dei principali aspetti progettuali

La realizzazione operativa del *Project Work* è stata concepita seguendo un approccio di tipo *end-to-end*, strutturando il flusso di lavoro in diversi moduli indipendenti e interconnessi tra loro. Proprio come Virgilio accompagna Dante nel suo complesso viaggio attraverso i vari gironi dell'Inferno, l'architettura del sistema si pone come "guida" del *dataset* dalla sua generazione artificiale, passando poi per le fasi di pulizia e vettorizzazione NLP, fino all'addestramento dei modelli predittivi e alla restituzione del risultato in un'interfaccia utente web reattiva e accessibile.

All'interno di questo documento saranno presenti solo le parti di codice che vale la pena analizzare. Per garantire la massima trasparenza, riproducibilità e verificabilità del codice condivido il link della *repository* GitHub principale. Inserisco anche i link utili per l'accesso alla *web app* e alla *repository* GitHub utile al suo mantenimento:

- Link repository principale:
  `https://github.com/gabrieleStagnitta/Project-Work`
- Link repository per la dashboard:
  `https://github.com/gabrieleStagnitta/Project-Work-Dashboard`
- Link web app dashboard: `https://project-work-vdxq.onrender.com/`

Scansiona il codice QR per accedere alla repository principale.

All'interno della *repository* principale è presente un file `README.md` che illustra nel dettaglio la struttura e l'organizzazione delle cartelle e dei file. I vari file *Jupyter Notebook* contengono già al loro interno una dettagliata descrizione delle varie fasi trattate al loro interno. Inoltre è sempre presente una cella di codice con all'interno il comando per l'installazione delle librerie utilizzate nel progetto.

2.3.1 Generazione Dataset elementare - `01_Generazione_Dataset_Elementare.ipynb`

In questo *Jupyter Notebook* è possibile consultare una soluzione elementare per la realizzazione del primo obiettivo del *Project Work*. Gli obiettivi specifici raggiunti in questa sezione sono:

- Creazione di un piccolo *dataset* sintetico (200–500 ticket testuali) con etichetta di categoria e priorità.
- Sintesi di ticket brevi (titolo + descrizione) con lessico tipico per Amministrazione, Tecnico, Commerciale.
- Etichette di priorità semplici basate su parole chiave (es. 'errore', 'fattura', 'bloccante').
- Esportazione CSV con colonne: `id, title, body, category, priority`.

Come prima cosa, è stato generato un file `.json` con all'interno un vocabolario da utilizzare per realizzare i ticket. Il JSON contiene una divisione per 3 tipologie di ticket, come da consegna: Amministrazione, Tecnico, Commerciale. Ogni categoria contiene a sua volta 3 liste, ovvero: Verbi, Sostantivi, Dettagli dei ticket (frasi o contesti specifici). Link al file: `https://github.com/gabrieleStagnitta/Project-Work/blob/main/input/lessico.json`

```
{
  "Amministrazione": {
    "verbi": [
      "sollecitare", "verificare", "restituire", "rettificare", "
         inviare",
      "ricevere", "assistere", "contabilizzare", "approvare", "
         stornare",
      "sollecitare", "archiviare", "sospendere"
    ],
    "sostantivi": [
      "la fattura", "il pagamento", "la nota di credito", "lo
         storno",
      "il rimborso", "l'estratto conto", "la distinta di bonifico
         ",
      "il sollecito di pagamento", "il giustificativo di spesa",
      "la nota spese"
    ],
    "dettagli": [
      "del mese scorso", "per il servizio cloud", "con importo
         errato",
      "non ancora pervenuto", "relativo al trimestre in corso",
      "mancante della firma digitale", "da inoltrare al
         commercialista",
      "per duplicazione interna"
    ]
  }
}
```

Figura 2.2: Estratto del contenuto del file lessico.json

Lo spezzone di codice a seguire mostra la logica che sta alla base della generazione delle frasi che popoleranno i ticket. All'interno della variabile lessico è stato precedentemente conservato il contenuto del JSON. All'interno del ciclo `for`, avviene invece la generazione di un singolo ticket: vengono estratti in maniera pseudocasuale tutti i componenti che faranno parte del ticket finale. Prima la

categoria, poi casualmente si sceglie se inserire o meno del rumore artificiale, aggiungendo parole prese da altre categorie (per provare ad evitare *overfitting*). Si scelgono i verbi, sostantivi, e i dettagli da utilizzare nel ticket, tutti parametri da inserire a dei formati preimpostati, che presentano delle frasi con una determinata struttura lessicale.

```
for i in range(num_ticket):
    cat_reale = random.choice(categorie)

    # C'e una probabilita del 15% che il ticket usi parole
       incrociate
    cat_verbo = cat_reale if random.random() > 0.15 else random.
       choice(categorie)
    cat_sost = cat_reale if random.random() > 0.15 else random.
       choice(categorie)
    cat_dett = cat_reale if random.random() > 0.15 else random.
       choice(categorie)

    v = random.choice(lessico[cat_verbo]["verbi"])
    s = random.choice(lessico[cat_sost]["sostantivi"])
    d = random.choice(lessico[cat_dett]["dettagli"])

    # Generazione Titolo (lista formati)
    oggetti = [
        f"{v.capitalize()} {s}",
        f"URGENTE: {v.capitalize()} {s}",
        f"Richiesta supporto - categoria: {cat_reale}",
        # ... altri formati ...
    ]
    title = random.choices(oggetti, weights=pesi_oggetti)[0]

    # Generazione Corpo (lista formati)
    formati = [
        f"Buongiorno,\ncon la presente si richiede cortesemente
           di {v} {s} {d}.\nRestiamo in attesa.",
        f"Ciao, per favore dovremmo {v} {s} {d} il prima
           possibile. Grazie!",
        # ... altri formati ...
    ]
    # Estrazione del corpo con i rispettivi pesi
```

Figura 2.3: Implementazione della generazione procedurale con pesi distribuiti e iniezione di rumore semantico

Infine, si attribuisce un livello di priorità, che dipende dalle parole sorteggiate al suo interno. Il tutto viene poi conservato in un file `.csv` di *output*.

Questo *script* basato su regole statiche si comporta già abbastanza bene, ma non ero ancora soddisfatto dell'*output* finale. La pseudo-randomizzazione che contraddistingue l'architettura rende i ticket innaturali e, nella maggior parte dei casi, molto ripetitivi. Per superare tale limite, ho deciso dunque di sfruttare la potenza dei modelli generativi, che verrà analizzato meglio nel prossimo *Jupyter Notebook*. Si può consultare un breve estratto nella figura 1.1 o dal GitHub.

2.3.2 Generazione Dataset con Gemma4:e2b - `02.1_Generazione_Dataset_Avanzato_Con_Gemma.ipynb`

Se l'approccio elementare ha permesso di immaginare la struttura che avrebbe dovuto impiegare il *dataset*, è solo con l'integrazione dei LLM che si è riusciti a conferire ai ticket quella complessità, imprevedibilità e sfumature lessicali che caratterizzano le reali comunicazioni tra esseri umani. L'implementazione di questo secondo modulo si è basata su scelte ben precise che ho trovato molto interessanti da implementare e che considero un grosso pilastro del progetto.

Invece di affidarsi ad API *cloud* esterne a pagamento (come ad esempio Gemini), l'intera *pipeline* di generazione è stata fatta girare localmente sul mio portatile tramite *client* `Ollama`. Questa scelta ha garantito il controllo totale del processo, l'assenza di costi o limiti di traffico (*rate-limiting*), dimostrando la fattibilità di un sistema di smistamento dati completamente isolato e sicuro.

Durante le fasi preliminari di test, l'utilizzo della versione base di `Gemma4` produce testi eccellenti ma richiede tempi di elaborazione insostenibili. Si può consultare un breve estratto del dataset nella figura 1.2 o dal GitHub.

La scelta di passare ad un modello più compatto come `Gemma4:e2b` ha rappresentato il perfetto compromesso: i tempi si sono ridotti a oltre 5 ticket al minuto, consentendo la generazione di un *dataset* bilanciato di 500 record in meno di due ore e permettendo molteplici sessioni di iterazione sul *prompt*, al solo costo di avere dei ticket meno complessi. Cosa che, come analizzeremo nella prossima sezione, in realtà si è rivelata anch'essa un punto a favore del progetto. Si può consultare un breve estratto del dataset nella figura 1.3 o dal GitHub.

La classica sfida nell'utilizzo di qualsiasi LLM per la creazione di banche dati è l'instabilità dell'*output*: i modelli tendono infatti ad aggiungere flussi di coscienza lunghi, ad aggiungere caratteri in *markdown*, emoji o formattazioni impreviste che creerebbero problemi durante la fase di generazione. Per eliminare il problema alla radice, il codice integra la libreria `Pydantic` per applicare degli *output* strutturati e la libreria `Ollama` per effettuare delle richieste con dei *prompt* parametrici. Questo permette inoltre di impostare un formato specifico per le risposte del modello (cosa che ha semplificato l'inserimento di ogni singolo ticket nel file `.csv` di *output*).

Nota: Quelli presenti nelle Immagini 2.4 e 2.5 non sono altro che prompt di esempio. I ticket consultabili nei dataset sono stati creati in vari batch, modificando periodicamente le restrizioni in base alle necessità.

La cella di codice consultabile dall'Immagine 2.6 sceglie, sorteggiando casualmente da delle liste create precedentemente, tutti i parametri da passare per la generazione del *prompt*. L'*output* di ogni iterazione sarà un singolo ticket, che viene infine conservato all'interno di un `.csv`.

```
class TicketSintetico(BaseModel):
    title: str = Field(
        description="Un titolo breve, specifico e realistico."
    )
    body: str = Field(
        description=(
            "Il corpo del ticket. Deve contenere una descrizione
               breve (1-2 frasi) del problema reale. "
            "NON usare placeholder, parentesi quadre o simboli di
                markup... "
            # [Altre istruzioni di validazione]
        )
    )
```

Figura 2.4: Esempio di struttura base dei parametri di generazione realizzata ereditando la classe BaseModel di Pydantic

```
def genera_ticket_con_gemma(categoria, priorita, esperienza,
   colloqualita, incipit, titolo, disturbo=False, typo=False):
    prompt_utente = f"""Genera un ticket di assistenza aziendale
       in lingua italiana con queste caratteristiche:
    - Categoria del ticket: {categoria}.
    #Altre caratteristiche e regole da seguire
    """
    try:
        response = client.chat(
            model='gemma4:e2b',
            messages=[{'role': 'user', 'content': prompt_utente
               }],
            format=TicketSintetico.model_json_schema(),
            options={
                'keep_alive': -1,
                'temperature': 0.4
            }
        )
        dati_ticket = json.loads(response['message']['content'].
           strip())
        return dati_ticket['title'], dati_ticket['body']
    except Exception as e:
        print(f"Errore durante la generazione locale: {e}")
        return None, None
```

Figura 2.5: Funzione della generazione di un ticket. Qui viene formattato il prompt per la creazione di un ticket e sottoposto all'LLM.

```
for i in range(0, da_fare):
    stringa_id = f"TK-{id_di_partenza + i}"

    # Estrazione casuale dei parametri di base
    categoria = random.choice(categorie_disponibili)
    priorita = random.choice(priorita_disponibili)
    esperienza = random.choice(livelli_esperienza)
    colloqualita = random.choice(livelli_colloquialita)
    attiva_typo = random.choices([True, False], weights=[20, 80])
       [0]

    # Scelta incipit e titolo basata sulla priorita (uso dei pesi
        per creare rumore)
    if priorita == "Alta":
        pool_scelta_titolo = random.choices([pool_titolo_urgenti,
            pool_titolo_normali], weights=[70, 30])[0]
        titolo = random.choice(pool_scelta_titolo)
        # Logica simile per priorita Media e Bassa

    disturbo = ""
    if random.randint(0,2) == 1: # 33% di probabilita di inserire
        un disturbo contestuale
        if categoria == "Tecnico":
            disturbo = f"L'utente specifica di lavorare presso: {
               random.choice(reparti_aziendali)}"
        # [Gestione disturbo per Amministrazione e Commerciale
           simile]

    print(f"Generazione in corso... [{categoria} - {priorita}]")

    # Chiamata all'LLM (funzione descritta precedentemente)
    titolo, corpo = genera_ticket_con_gemma(categoria, priorita,
       esperienza, colloqualita, incipit, titolo, disturbo,
       attiva_typo)

    if titolo and corpo:
        singolo_ticket = {
            "id": [stringa_id],
            "title": [titolo],
            "body": [corpo],
            "category": [categoria],
            "priority": [priorita]
        }
        df_singolo = pd.DataFrame(singolo_ticket)
```

Figura 2.6: Ciclo responsabile della generazione di n ticket da generare

2.3.3 Generazione Dataset con architettura multi-agent - `02.2_Generazione_Dataset_Architettura_Actor_Critic.ipynb`

Questa nuova versione presenta un'architettura Multi-Agente chiamata in letteratura *Actor-Critic*. Invece di sperare che l'LLM faccia tutto giusto al primo colpo, si creano dei sistemi in cui i modelli "ragionano" e si correggono a vicenda. La struttura del codice è molto simile, l'unica aggiunta consiste nella presenza di una nuova classe utile a formattare il ticket revisionato. Nello specifico, viene aggiunto il campo `modifiche_applicate`, che non sarà altro che un booleano che indica se il revisore ha fatto o meno delle modifiche.

```
class TicketRevisionato(BaseModel):
    titolo: str = Field(
        description="Il titolo del ticket corretto e ripulito da
           markdown o placeholder."
    )
    corpo: str = Field(
        description=(
            "Il corpo del ticket corretto e ripulito dai soli
               artefatti dell'IA. "
            # [Istruzioni sul mantenimento del tono emotivo e
               informale omesse per brevita]
        )
    )
    modifiche_effettuate: str = Field(
        description="Descrivi in una frase breve quale tipo di
           rumore o imperfezione hai aggiunto."
    )
```

Figura 2.7: Esempio di struttura base dei parametri di revisione realizzata ereditando la classe BaseModel di Pydantic

Un'ulteriore modifica consiste nell'inserimento di un nuovo parametro nella generazione base dei ticket, ovvero il genere del mittente. Poiché i ticket generati da Gemma presentavano esclusivamente mittenti maschili, ho implementato una variabile casuale aggiuntiva per definire il sesso della persona richiedente.

2.3.4 Pipeline Machine Learning - `03_Pipeline_Machine_Learning.ipynb`

In questo *Jupyter Notebook* viene realizzato il secondo obiettivo del *Project Work*, ovvero lo sviluppo della *pipeline* ML. In particolare, è possibile consultare i processi di:

- Lettura dei dati ed EDA
- *Preprocessing* (pulizia base del testo)
- Modello classificazione categoria

- Modello stima priorità
- Valutazione dei modelli
- Esportazione dei modelli per la creazione della *dashboard*

Come sempre, all'interno del *notebook* vengono analizzati nel dettaglio tutti gli elementi utilizzati. In questo documento riporto solo un estratto dell'EDA per il *dataset* che ha mostrato il risultato più bilanciato, ovvero quello realizzato con `l'architettura Actor Critic`.

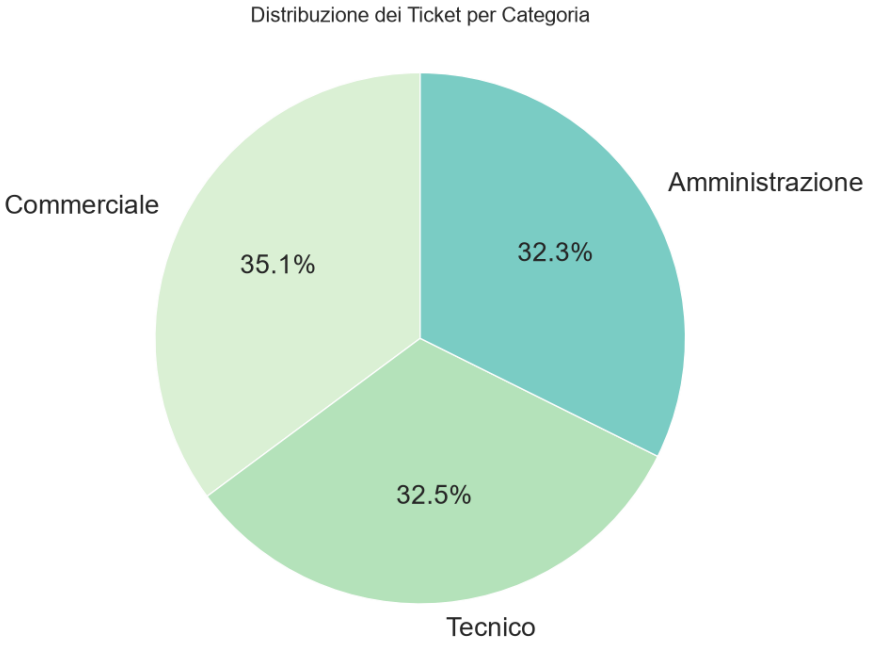


Figura 2.8: Grafico a torta della distribuzione dei ticket per categoria

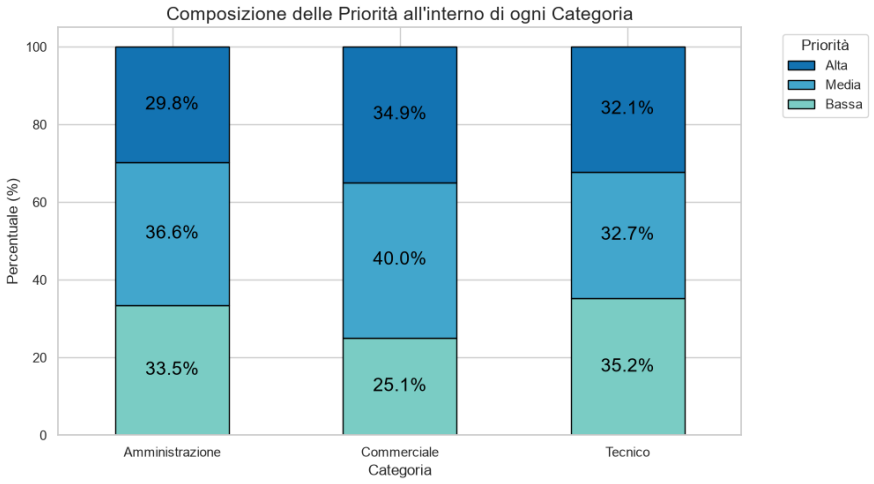


Figura 2.9: Tabella incrociata della composizione priorità nelle varie categorie

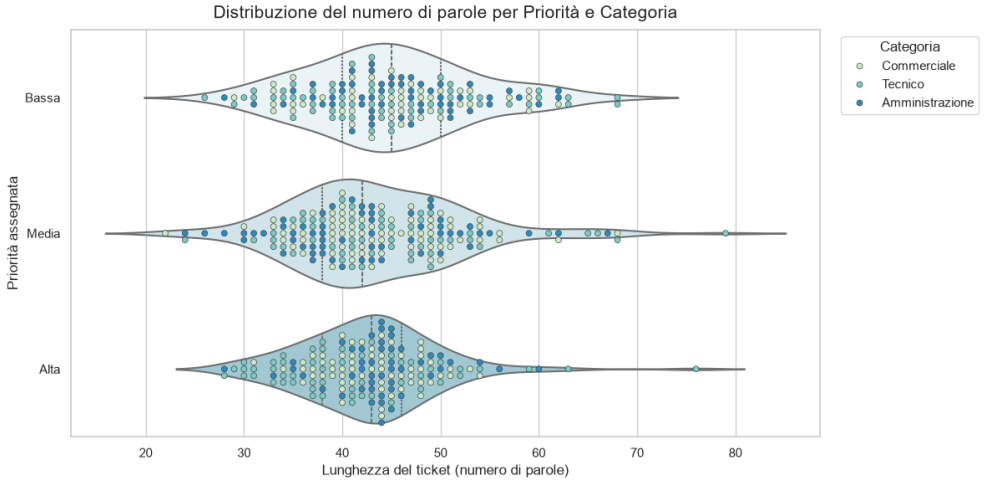


Figura 2.10: Violin plot + Swarm Plot della distribuzione del numero di parole per Priorità e Categoria

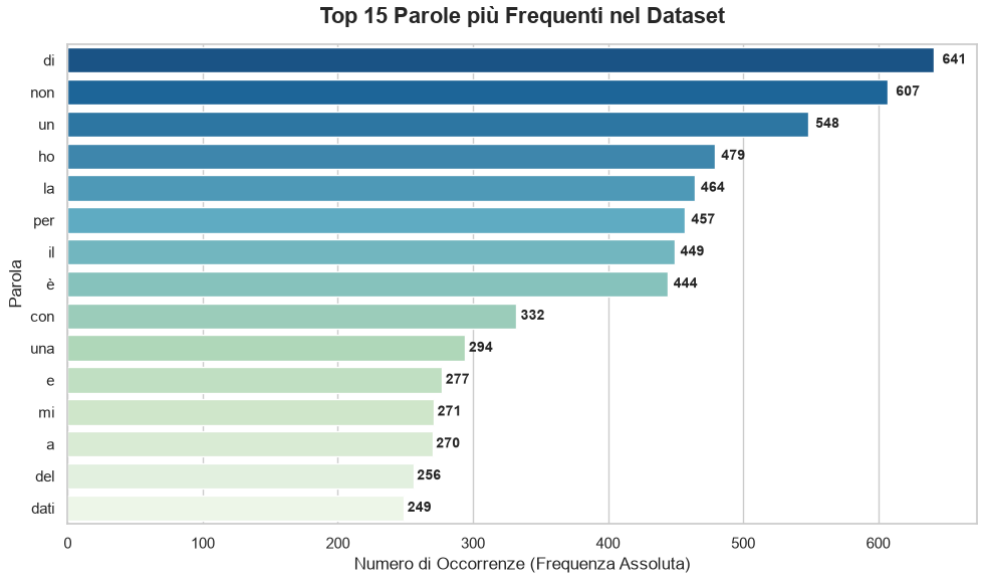


Figura 2.11: Grafico a barre delle 15 parole più frequenti nel dataset

Come è possibile notare dalle figure 2.8, 2.9 e 2.10, `Gemma4:e2b` ha rispettato il vincolo di lunghezza dei ticket datogli in quanto, escludendo gli *outlier*, il numero di parole medio si aggira tra le 30 e le 40 per ticket. Inoltre, si può anche notare come vi sia un ottimo bilanciamento della distribuzione dei ticket sia per categoria, che per priorità.

Dalla figura 2.11 invece vediamo come le parole più utilizzate sono prettamente *stopword*, ovvero parole che servono a costruire una frase, senza fornire ad essa un significato. Nonostante questo possa sembrare un "rumore", il vettorizzatore che verrà presentato a breve risolve autonomamente questo problema, dato che il suo compito è quello di assegnare matematicamente un peso a tutte le parole; in particolare, questo abbassa in automatico il peso delle parole che compaiono nella

maggior parte dei ticket (come appunto le *stopword*). In ogni caso, verrà passato al vettorizzatore una lista di supporto contenente *stopword* italiane proprio per aiutarlo in questo compito (figura2.3.4)

```
stop_italiane = ['di', 'a', 'da', 'in', 'con', 'su', 'per', 'tra'
    , 'fra', 'il', 'lo', 'la', #... [altre stopword] ...
]
```

Figura 2.12: Lista di supporto per le stopword

Facendo un piccolo passo in avanti, anticipo che questa operazione farà scendere leggermente la precisione nelle previsioni, ma di una percentuale altamente ignorabile. Considerando ciò che abbiamo appena analizzato, questo è un compromesso perfettamente accettabile al fine di rimuovere il rumore e fare concentrare gli algoritmi solo sulle parole realmente importanti per i ticket.

Ritorniamo indietro e parliamo della procedura di *preprocessing* che si basa prevalentemente su operazioni eseguite utilizzando la libreria `regex`. La necessità dell'implementazione della funzione `remove_key_word` è maturata a seguito di diverse fasi di *testing* fallimentari con i *dataset* realizzati da Gemma. Ho notato che, nonostante avessi specificato nel *prompt* di base di non usare parole come "tecnico, commerciale, amministrazione", queste continuavano ad essere presenti ed essere predominanti nei ticket. Lo scopo di questa funzione è dunque quello di rimuovere le radici delle parole che svelano la categoria, ovvero togliere parole come commerciale, amministrazione, amministrativo, ecc, in modo da farle diventare irrilevanti.

```
def remove_key_word(testo):
    testo = testo.lower()
    # Rimuove le radici delle parole che svelano la categoria
    testo = re.sub(re.compile(r'\b(tecnic\w*|commerci\w*|
       amministr\w*|fattur\w*)\b'), '', testo)
    return testo
```

Figura 2.13: Funzione remove_key_word

Superata la fase di Analisi Esplorativa (EDA) e ultimata la pulizia di base del testo, si è posta la necessità di convertire i ticket in matrici numeriche, così da renderle comprensibili per gli algoritmi di apprendimento supervisionato. A tale scopo, la *pipeline* integra il modulo `TfidfVectorizer` della libreria Scikit-Learn.

Rispetto a un semplice conteggio delle frequenze, la misura statistica TF-IDF permette di pesare l'importanza di ogni singolo termine, valorizzando le parole chiave identificative di un reparto (es. "bonifico", "router", "licenza") e penalizzando i termini generici comuni all'intero *dataset* (come le *stopword* di cui parlavamo prima).

Ho trovato molto interessante la matematica dietro questo processo. Senza entrare troppo nel dettaglio, il peso TF-IDF di una parola viene calcolato moltiplicando la frequenza del termine (TF) per la frequenza inversa del documento

(IDF), ovvero la misura della rarità di un termine in tutti i documenti. Citando e semplificando la spiegazione matematica dell'algoritmo, consultabile da Wikipedia:

$$\mathrm{TF}(t, d) = \frac{\text{numero di volte che } t \text{ appare in } d}{\text{numero totale di parole in } d}$$

$$\mathrm{IDF}(t) = \ln\left(\frac{\text{numero totale dei documenti}}{\text{numero di documenti che contengono } t}\right)$$

$$\text{TF-IDF}(t, d, D) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t, D)$$

Figura 2.14: Formule matematiche per il calcolo del peso TF-IDF

Dove t rappresenta una parola, d il documento di appartenenza (in questo caso, un singolo ticket). Per numero totale dei documenti in questo progetto si intende invece il numero di ticket totali.

Nota: `TfidfVectorizer` di Scikit-Learn implementa una variante leggermente diversa ma più complessa delle formule matematiche, mantenendo però il ragionamento di base.

Per il *task* di classificazione ho utilizzato due approcci classici e uno ideato appositamente:

- Random Forest Classifier
- LinearSVC
- Approccio Gerarchico

Come ci si può aspettare, i diversi *dataset* hanno portato diversi risultati, per ognuno di essi viene posta un'accurata valutazione dei risultati (guarda la sezione "Fasi di lavoro[...]" per un'analisi più accurata). Il risultato più stabile per la classificazione delle categorie lo ha dato l'algoritmo Linear SVC in quasi tutti i dataset.

Per quanto invece riguarda la valutazione della priorità, l'approccio vincente è stato decisamente quello dell'addestramento gerarchico. Questa logica consiste nel realizzare inizialmente il modello di classificazione categorie per poi addestrare in maniera verticale i modelli di previsione della Priorità per ogni reparto.

```
mod_cat_gerarchico = LinearSVC(random_state=42, dual=False)
mod_cat_gerarchico.fit(X_train_cat_gerarchico, train_df['category
    '])

# Predizione delle categorie (Tecnico, Amministrazione,
    Commerciale)
test_df['pred_category'] = mod_cat_gerarchico.predict(
    X_test_cat_gerarchico)

# Addestramento modelli di priorita per la categoria
modelli_priorita_reparto, vectorizers_priorita_reparto = {}, {}

for cat in train_df['category'].unique():
    # Filtra i dati del TRAIN SET per la categoria corrente
    train_sottogruppo = train_df[train_df['category'] == cat]

    vec_prio = TfidfVectorizer(sublinear_tf=True, min_df=2,
        ngram_range=(1, 2), stop_words=stop_italiane)
    X_train_prio_sub = vec_prio.fit_transform(train_sottogruppo['
        testo_completo'])

    clf_prio_sub = LinearSVC(random_state=42, dual=False,
        class_weight='balanced')
    clf_prio_sub.fit(X_train_prio_sub, train_sottogruppo['
        priority'])

    modelli_priorita_reparto[cat.lower().strip()] = clf_prio_sub
    vectorizers_priorita_reparto[cat.lower().strip()] = vec_prio

# Pipeline gerarchica del test set
priorita_predette_finali = []

for idx, riga in test_df.iterrows():
    categoria_stimata = str(riga['pred_category']).lower().strip
        ()
    testo_ticket = riga['testo_completo']

    if categoria_stimata in vectorizers_priorita_reparto:
        vettorizzatore_specifico = vectorizers_priorita_reparto[
            categoria_stimata]
        modello_specifico = modelli_priorita_reparto[
            categoria_stimata]

        testo_vettorizzato = vettorizzatore_specifico.transform([
            testo_ticket])
        prio_stimata = modello_specifico.predict(
            testo_vettorizzato)[0]
    else:
        prio_stimata = riga['priority']

    priorita_predette_finali.append(prio_stimata)

test_df['pred_priority'] = priorita_predette_finali
```

Figura 2.15: Approccio gerarchico alla previsione della categoria e priorità

La cella di codice in figura 2.16, nonostante le sue poche righe, ha un compito molto importante: salvare i modelli scelti in dei file, che verranno utilizzati successivamente per il funzionamento della *dashboard*.

```
import joblib

# salviamo i componenti chiave del sistema
joblib.dump(mod_cat_gerarchico, 'modelli/m_categoria.pkl')
joblib.dump(vec_cat_gerarchico, 'modelli/v_categoria.pkl')

# salviamo anche i dizionari dei modelli verticali delle priorita
joblib.dump(modelli_priorita_reparto, 'modelli/m_priorita.pkl')
joblib.dump(vectorizers_priorita_reparto, 'modelli/v_priorita.pkl
    ')
```

Figura 2.16: Script responsabile al salvataggio di modelli e vettorizzatori

Il *notebook* si conclude con un'analisi dei dati conclusivi con dei grafici a barre orizzontali che mostrano, in base alla categoria o alla priorità, le parole che più influenzano i modelli (il peso delle parole attribuito dai vettorizzatori analizzati precedentemente). Risulta possibile una consultazione nella pagina a seguire (figura 2.17 e 2.18)

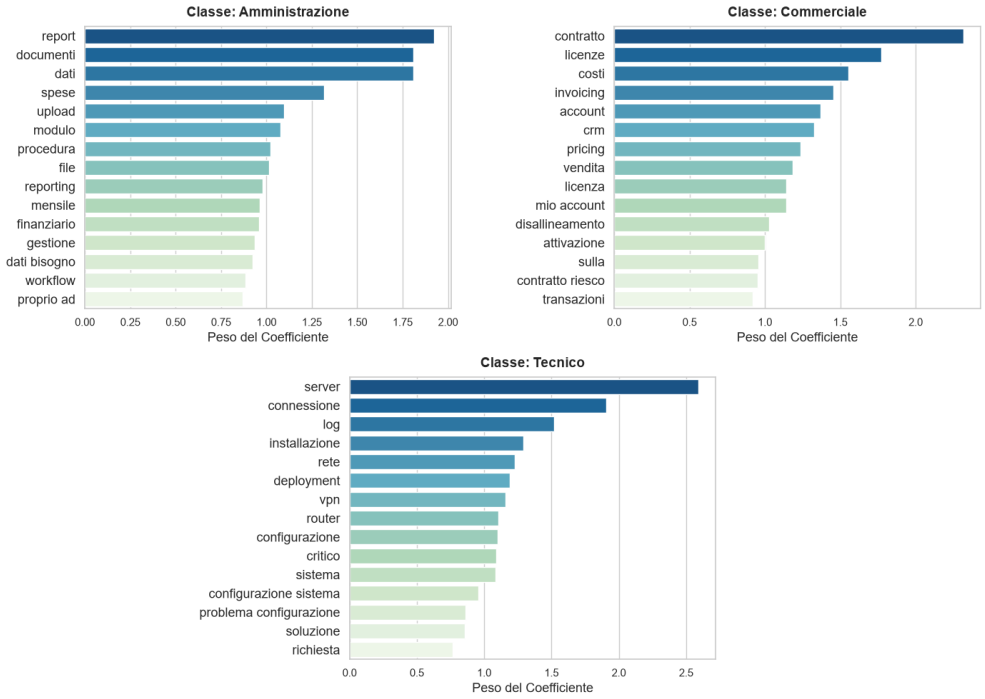


Figura 2.17: Top 15 Parole più influenti per la categoria

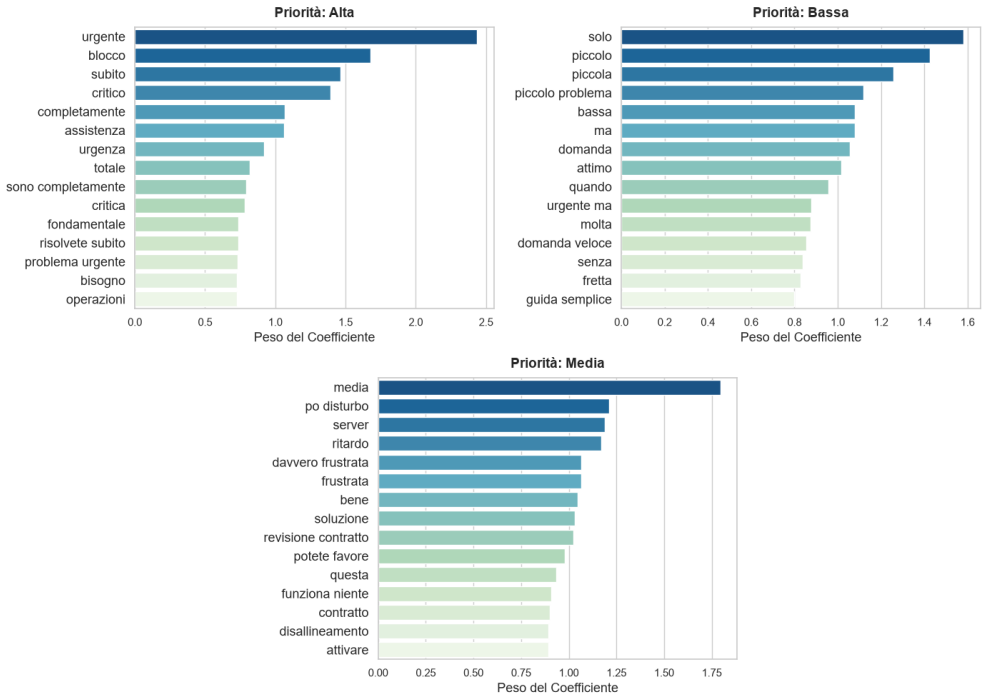


Figura 2.18: Top 15 Parole più influenti per la priorità

2.3.5 Dashboard - `Dashboard.py`

In quest'ultimo modulo avviene la trasformazione della *pipeline* di *back-end* in un'interfaccia utente interattiva, moderna e direttamente fruibile da un potenziale personale. I dati vengono analizzati caricando i modelli e i *vectorizer*. La funzione `analizza_singolo()` è responsabile dell'analisi di un singolo ticket inserito da un operatore e dell'aggiornamento dell'interfaccia mostrando i risultati dell'analisi.

```
try:
    vec_categoria = joblib.load('modelli/v_categoria.pkl')
    model_cat = joblib.load('modelli/m_categoria.pkl')
    # caricamento dei dizionari per l'approccio gerarchico
    dict_vec_priorita = joblib.load('modelli/v_priorita.pkl')
    dict_mod_priorita = joblib.load('modelli/m_priorita.pkl')
    print("Modelli caricati con successo!")
```

Figura 2.19: Caricamento modelli

```
def analizza_singolo():
    titolo = input_titolo.value
    corpo = input_corpo.value
    # preprocessing
    testo_pulito = clean_text(titolo + " " + corpo)
    # predizione categoria (fase 1)
    X_cat = vec_categoria.transform([testo_pulito])
    categoria = model_cat.predict(X_cat)[0]
    # chiave di ricerca per il dizionario
    cat_key = str(categoria).lower().strip()
    # predizione priorita (fase 2 - gerarchica)
    if cat_key in dict_vec_priorita:
        vec_pri_specifico = dict_vec_priorita[cat_key]
        mod_pri_specifico = dict_mod_priorita[cat_key]

        X_pri = vec_pri_specifico.transform([testo_pulito])
        priorita = mod_pri_specifico.predict(X_pri)[0]

        parole_pri = get_top_words(testo_pulito,
           vec_pri_specifico, mod_pri_specifico)
    # aggiornamento UI
    lbl_categoria.set_text(categoria.upper())
    lbl_priorita.set_text(priorita.upper())

    parole_cat = get_top_words(testo_pulito, vec_categoria,
       model_cat)
```

Figura 2.20: Logica dell'analisi del singolo ticket. Le funzioni clean_text e get_top_words sono le stesse analizzate anche nel notebook 03.

Superati i limiti visivi e strutturali riscontrati inizialmente con librerie *desktop* tradizionali come `Tkinter`, lo sviluppo si è indirizzato verso il *framework* `NiceGUI`. L'applicazione `Dashboard.py` è stata progettata privilegiando un'esperienza d'uso essenziale e priva di attriti cognitivi:

- Area di testo pulita in cui l'operatore può incollare o digitare il corpo della comunicazione non strutturata.
- Restituzione istantanea della Categoria prevista e della Priorità suggerita.
- Top 5 parole influenti, che permette di validare o correggere il suggerimento della macchina.
- Area per l'elaborazione *batch* dei ticket fornendo in *input* un file `.csv` con all'interno le colonne "title" e "body".

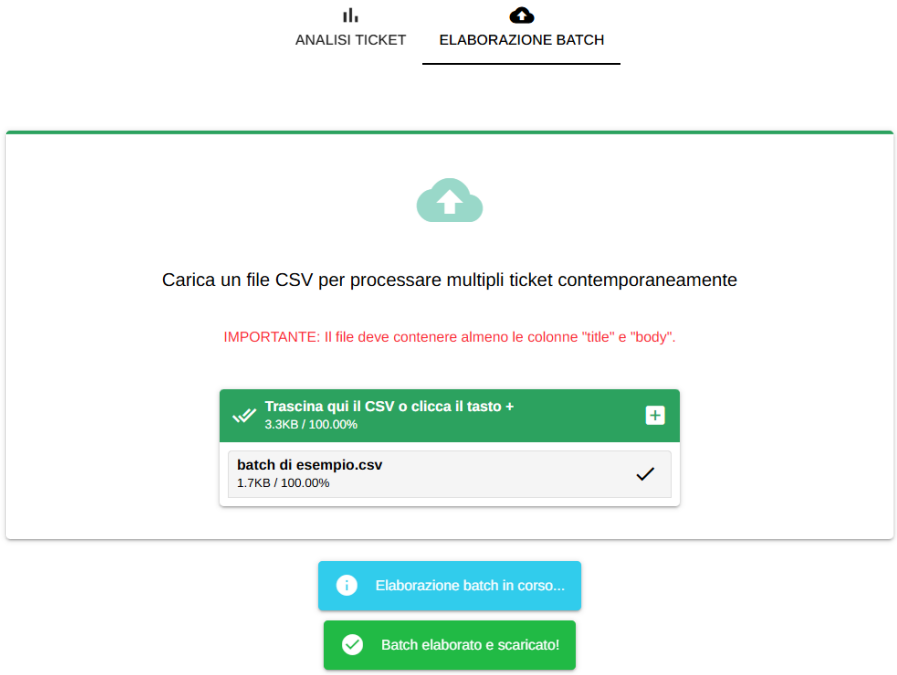


Figura 2.21: Interfaccia della dashboard - sezione "elaborazione batch"

Infine, per soddisfare il requisito di riproducibilità e rapida esecuzione dell'elaborato, il codice sorgente della *dashboard* è stato collegato ad un ulteriore *repository* GitHub e messo in produzione *cloud* tramite la piattaforma Render al seguente link. Piccolo dettaglio finale a scopo puramente estetico: è possibile visualizzare la *web app* in *dark mode* o in *light mode*. Uno screenshot e il link della web app è disponibile a pagina 15 (grazie alla figura 1.8).

2.4 Campi di applicazione

L'architettura di *Machine Learning* applicata allo *Smart Ticketing* realizzata nel *project work*, è stata concepita con una logica modulare. Questa caratteristica ne consente una rapida adozione e reingegnerizzazione in molteplici contesti organizzativi reali, ponendosi come strumento di supporto alle decisioni per ottimizzare i flussi di lavoro aziendali.

Ambiti operativi di implementazione:
La versatilità della *pipeline* permette di applicare il modello in diversi settori, adattando semplicemente il vocabolario del dataset di addestramento al dominio di riferimento:

- **Help Desk IT e Assistenza Tecnica Aziendale:** ambito naturale di applicazione del prototipo. In organizzazioni strutturate (ad esempio società di consulenza informatica), il sistema può smistare istantaneamente le segnalazioni dei dipendenti o dei clienti tra i vari livelli di supporto, assegnando automaticamente una priorità elevata a incidenti critici che bloccano l'operatività.
- **Customer Care:** immaginato nel mio contesto lavorativo attuale (scuola online di *Coding* per bambini), dove giornalmente il reparto di assistenza clienti riceve comunicazioni non strutturate sotto forma di email o messaggi Whatsapp da parte dei genitori dei nostri studenti. Il modello potrebbe catalogare le richieste senza intervento umano e spedirle direttamente ai reparti di competenza, ad esempio *Customer Success* (per info sulle offerte o sui pagamenti), Supporto Tecnico (per richieste di aiuto su installazioni e varie) o Insegnanti (per informazioni sulle lezioni o su richieste di *feedback*) o *Sales* (per informazioni riguardo iscrizioni o lezioni di prova).



Figura 2.22: Mockup di un modello di Customer Care, dove sono presenti delle richieste verosimili da parte di genitori immaginari

- **Call Center:** l'architettura potrebbe essere inserita come primo filtro intelligente nel mondo dei *call center* di una qualsiasi tipologia di attività: dai grandi operatori telefonici alle piccole aziende agricole locali. Abbinando la *pipeline* di *Machine Learning* a un processo di *Speech-to-Text*, il modello lavorerebbe come centralino telefonico intelligente. In questo scenario, è possibile immaginare un cliente descrivere verbalmente la propria necessità: il sistema trascrive la voce in testo, trasformandolo nell'equivalente dei ticket esaminati nel *Project Work*. Una volta eseguita l'analisi della richiesta, l'algoritmo procede nell'instradamento in tempo reale della chiamata all'operatore del reparto di competenza, mandando anche una trascrizione della richiesta per anticipare la segnalazione.

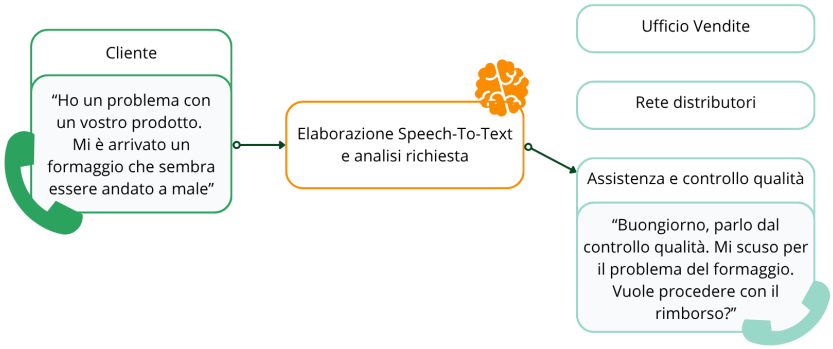


Figura 2.23: Mockup di un modello di Call Center per un'azienda del settore primario. Le chiamate vengono smistate tra gli operatori in base al reparto di competenza.

Vantaggi strategici e operativi:

L'adozione di un sistema di *Triage* automatico basato su *Machine Learning* introduce diversi benefici, tra cui maggiore competitività ed efficienza organizzativa:

- **Abbattimento del tempo medio di risposta:** l'eliminazione del collo di bottiglia causato dallo smistamento manuale riduce drasticamente il tempo che passa tra l'apertura e la presa in carico di un ticket.
- **Riduzione del tasso di errore e/o della soggettività:** lo smistamento manuale rischia di essere influenzato dalle interpretazioni arbitrarie di un problema da parte di un essere umano che spesso non è neanche esperto dell'argomento in questione. Un modello di *Machine Learning* addestrato invece garantisce criteri di valutazioni uniformi, oggettivi e replicabili nel tempo.

- **Ottimizzazione delle risorse e valorizzazione del capitale umano:** questo non implica togliere lavoro agli esseri umani, bensì sgravare il personale da mansioni puramente ripetitive e alienanti, dando la possibilità di concentrarsi solo sulla risoluzione dei problemi reali.

- **Trasparenza e supporto all'operatore:** grazie all'integrazione di un software di supporto, come ad esempio una *dashboard* interattiva, il sistema risulta facile da comprendere anche per operatori non specializzati in tutti i settori. L'unico compito sarebbe ricevere un suggerimento e scegliere se accettarlo, mantenendo comunque il controllo finale sulla supervisione e sulla validazione del processo.

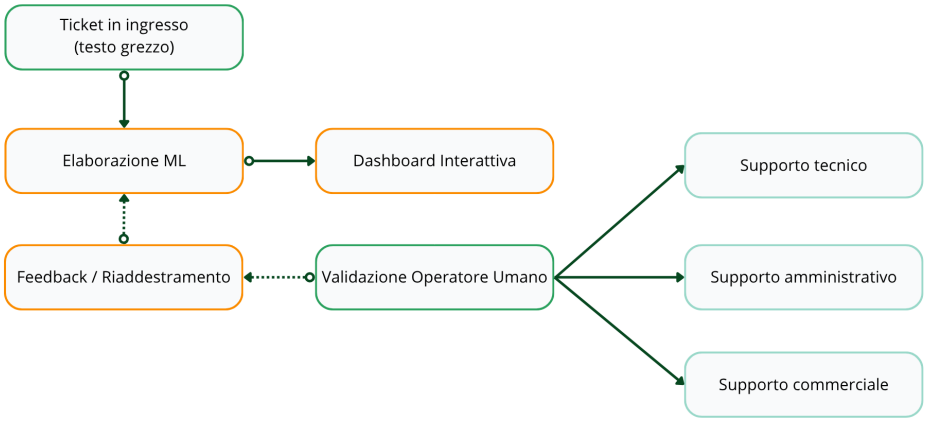


Figura 2.24: Esempio di flusso di gestione ticket con Machine Learning. In verde i processi umani/preesistenti, in arancione innovazione Machine Learning e in azzurrino output aziendale.

2.5 Valutazione dei risultati

Concludo l'elaborato con un'analisi critica dei risultati ottenuti da questo prototipo: partiamo dalla generazione dei dati.

L'utilizzo di LLM per la generazione di *dataset* è un pilastro importante del progetto che ne facilita la scalabilità in molteplici settori. Nonostante io mi sia soffermato solo sull'analisi della famiglia di modelli `Gemma4`, in realtà la *pipeline* è applicabile ad un qualsiasi modello di intelligenza artificiale utilizzabile con `Ollama`. Come ormai tutti sanno, ogni modello è specializzato in qualcosa (Claude nel *Coding*, Perplexity per ricerche approfondite, Gemini per il *general purpose*, ecc...). Questo implica che si possa scegliere un modello specifico in base al contesto in cui ci si trova.

Un limite che però ritengo da considerare è capire quanto la mia analisi sia corretta: il *dataset* con il modello standard di `Gemma4` ha dato risultati scadenti realmente a causa dei pochi ticket inseriti e alla complessità delle frasi generate?

Per risolvere questo potenziale problema, oltre all'ovvio tentativo di generare un *dataset* di dimensione maggiore, ho ragionato su due potenziali soluzioni: modifiche dei parametri chiave utilizzati e utilizzo di modelli di *Deep Learning* avanzati.

Nella *pipeline* attuale, i parametri utilizzati sono abbastanza generici e standard, ma potrebbero essere modificati per massimizzare la generalizzazione, ad esempio:

- **Random Forest:** Esplorazione della profondità massima (`max_depth`) per prevenire l'*overfitting* sui testi di *training*, e l'ottimizzazione del numero minimo di campioni per foglia (`min_samples_leaf`) per migliorare le regole decisionali.
- **LinearSVC:** Variazione del parametro di regolarizzazione (`C`). Un valore di `C` ottimizzato potrebbe permettere di trovare il giusto compromesso tra un margine di separazione ampio (che generalizza meglio) e la corretta classificazione dei ticket più ambigui nel *training set*.

Il secondo approccio consisterebbe nel superamento dell'approccio TF-IDF attraverso l'utilizzo di BERT (*Bidirectional Encoder Representations from Transformers*), un modello di *Deep Learning* sviluppato da Google e basato sull'architettura *Transformer*. L'integrazione di questa architettura potrebbe garantire due netti miglioramenti:

- **Risoluzione delle ambiguità:** BERT è in grado di comprendere il significato del termine in base al contesto in cui si trova. Es: distinguerebbe la frase "reset del router" dalla frase "reset del contratto".
- **Comprensione semantica profonda:** Per sua natura questo modello riesce a riconoscere sinonimi o termini affini (es: "urgente", "critico", "immediato", ecc...). Questo permetterebbe di incrementare non di poco la precisione sulla classificazione delle priorità, che è l'area in cui il TF-IDF ha mostrato i margini di miglioramento più ampi.

Per quanto riguarda l'interfaccia utente, l'attuale *dashboard* è un prototipo che permette unicamente l'inserimento manuale dei ticket, limitandone le potenzialità di utilizzo in un contesto lavorativo reale.

I futuri sviluppi potrebbero concentrarsi sul collegamento diretto ai canali di comunicazione aziendali (email o canali di messaggistica), permettendo al software un'analisi in tempo reale delle email ricevute, magari affiancandolo con l'aggiunta di un timer visivo associato al livello di priorità predetto, che indichi all'operatore il tempo massimo di presa in carico.

Infine, permettere all'operatore di fornire un *feedback* (confermando o correggendo la previsione) garantirebbe l'instradamento corretto e consentirebbe di raccogliere nuovi ticket realistici per alimentare l'apprendimento continuo (come discusso nei campi di applicazione)

Ringraziamenti

Giunto al termine di questo percorso accademico, desidero dedicare un pensiero a coloro che hanno incrociato il mio cammino in questi anni.

Rivolgo il primo pensiero alla mia famiglia: ai miei genitori e ai miei nonni. Sono cresciuto a metà tra due mondi, dividendo la mia infanzia tra i pomeriggi in campagna e le giornate passate al mare o in montagna. Alla fine credo di aver preso un po' dall'una e un po' dall'altro. Vi ringrazio per le radici che mi avete dato e per aver costruito lo scenario dei miei primi ricordi, il punto esatto da cui è partito tutto il mio percorso.

Un sincero ringraziamento va agli amici. Sia per i momenti di confronto che sono stati preziosi durante la creazione e la stesura di questo progetto, sia per la vostra presenza che è invece stata LETTERALMENTE di conforto nello stesso periodo (chi vuol capire capisca). Come disse una volta un vecchio saggio: "Du Bist Gut Genug".

Un grazie speciale va anche al team di Coding Giants. Mi avete fatto capire che gli ambienti di lavoro non devono per forza essere tossici. In questi anni sono passato dall'essere l'insegnante alle prime armi che rompeva costantemente le scatole alla sua protrainer per chiedere aiuto ("le domande che sono booleane e le risposte che sono stringhe"), a poter guidare in prima persona chi, come me, iniziava ad affacciarsi a questo mestiere. Per i materiali aggiuntivi, per il bot Gianni, per il nostro CRM che cambia in continuazione proprio come le scale di Hogwarts, per le serate gaming di Super Mario e, più in generale, per il confronto quotidiano che ha reso molto più leggera la routine in parallelo allo studio.

Il ringraziamento più speciale è dedicato ai miei studenti (sia i grandi che i "miei bimbi" dei corsi di coding). Il vostro entusiasmo, la vostra genuina curiosità e la voglia di scoprire sempre cose nuove hanno rappresentato per me una fonte di ispirazione costante, ricordandomi ogni giorno il vero valore dell'apprendimento continuo. Tanti momenti in cui, soprattutto con i piccoli, ho rischiato di perdere la sanità mentale peggio che per l'esame di Analisi Matematica (la classe dei mille traumi più uno), ma anche tanti momenti che non dimenticherò mai; non saprei contarli ma sicuramente più di sessantasette.

E infine, un ringraziamento a chi pensa di non potercela fare, a chi ha avuto paura di cadere, a chi un giorno si guarderà allo specchio e dirà: “Ce l’ho fatta”.

A noi tutti.

Bibliografia

Burkov, A. (2019). *The Hundred-Page Machine Learning Book*. Andriy Burkov.

Jurafsky, D., & Martin, J. H. (Draft). *Speech and Language Processing: An introduction to natural language processing, computational linguistics, and speech recognition* (3rd ed.). Disponibile su: `https://web.stanford.edu/~jurafsky/slp3/`

McKinney, W. (2010). Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51–56).

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Sitografia

Ferdinand, B. (2023). *NiceGUI: A nice HTML user interface in Python based on Vue.js and Tailwind CSS*. `https://github.com/zauberzeug/nicegui`

Google DeepMind. (2025). *Gemma 4 Model Card: Open-Weight Models Specifications*. Disponibile su: `https://ai.google.dev/gemma`

Max1234-Ita. (2020). *Repository GitHub di supporto per dataset testuali e anagrafiche italiane*. `https://github.com/Max1234-Ita`

Ollama Team. (2023). *Ollama: Get up and running with large language models locally*. `https://github.com/ollama/ollama`

Python Packaging Authority (PyPI). (2026). *The Python Package Index: Documentazione delle librerie software*. `https://pypi.org/`

Snowflake Fundamentals. (2025). *Principi di funzionamento e architettura degli algoritmi di Ensemble Learning: Random Forest*.